SECURITY · ARCHITECTURE

CRM Sync — Auth Pipelines

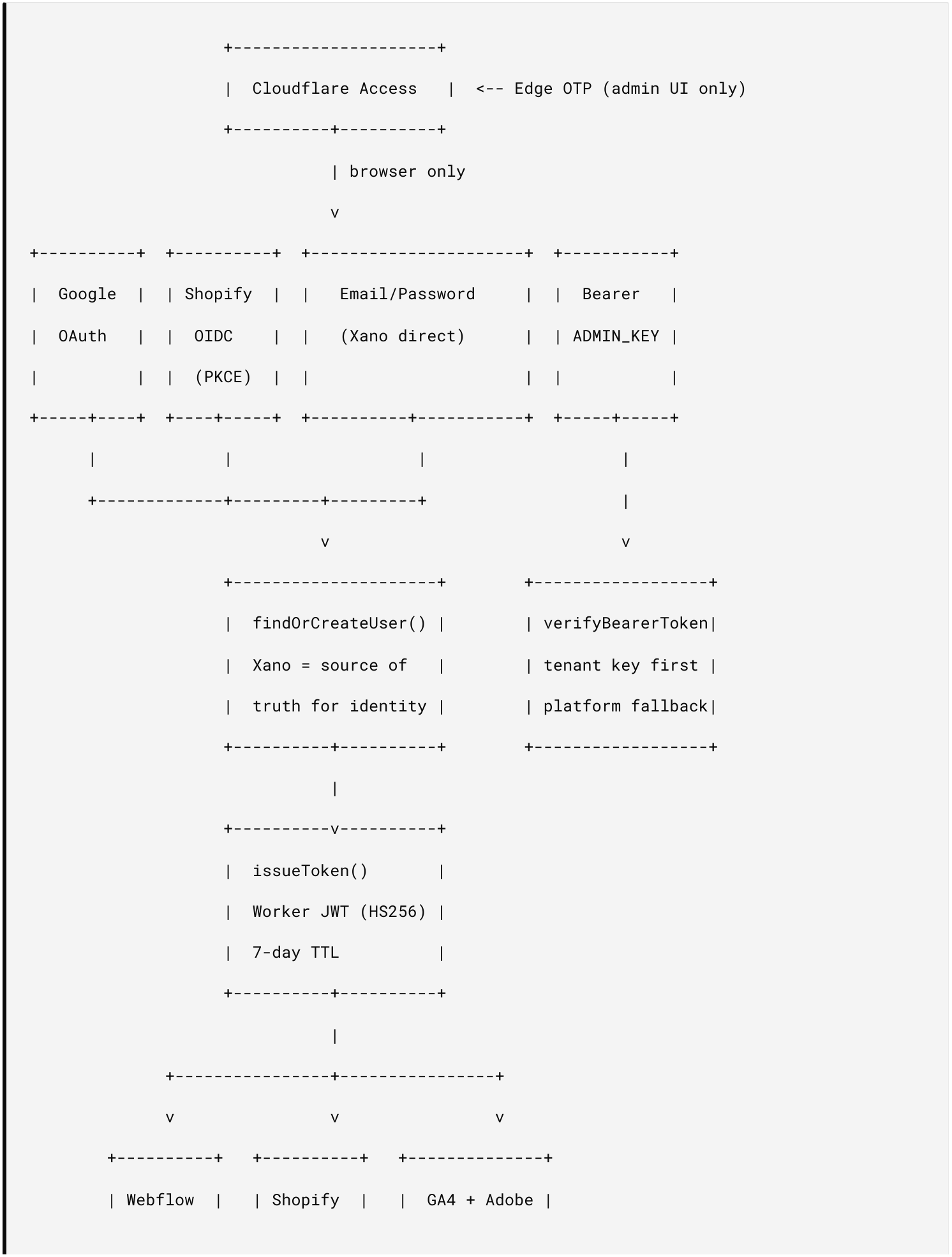
The **four independent authentication pipelines** behind CRM Sync — bearer tokens, JWT sessions, HMAC-signed webhooks, and Cloudflare Access — for engineering, security auditors, and compliance review.

For: **Engineering, security auditors, and compliance teams reviewing authentication architecture**

Date: **2026-05-19**

Overview

CRM Sync uses four independent authentication pipelines that converge at a single identity layer (Xano), then fan out to a shared real-time sync pipeline. Cloudflare Zero Trust operates as an edge-level network gate separate from the API auth stack.


Cloudflare Access
<-- Edge OTP (admin UI only)
browser only
Google OAuth
Shopify OIDC (PKCE)
Email/Password (Xano direct)
Bearer ADMIN_KEY
findOrCreateUser()
Xano = source of truth for identity
verifyBearerToken
tenant key first
platform fallback
issueToken()
Worker JWT (HS256)
7-day TTL
Webflow
Shopify
GA4 + Adobe

```
        | CMS sync |   | tags +   |   |  user props  |
        |          |   | metafield|   |  + XDM event |
        +----------+   +----------+   +--------------+
```

Pipeline 1: Cloudflare Zero Trust (Admin Layer)

Scope: Admin UI pages only (`/setup`, `/settings`, `/onboarding`). No API-level token validation in worker code.

```
Browser --> /setup, /settings, /onboarding
  |
  +-- Cloudflare Access edge intercept
  |     +-- OTP email verification (kcoop.cloudflareaccess.com)
  |     +-- Whitelisted: ysl@ysl150.com, *@story-story.ai
  |
  +-- verifyAdminKey(env, url)
  |     +-- ?key= query param OR ADMIN_KEY env var
  |
  +-- No token passed to Xano -- edge-only gate
```

What it protects

ROUTE	AUTH METHOD
`GET /setup`	Cloudflare Access OTP + `?key=` admin key
`GET /settings`	Cloudflare Access OTP + `?key=` admin key
`GET /onboarding`	Cloudflare Access OTP + `?key=` admin key

What it does NOT protect

All API endpoints (`/auth/*`, `/admin/*`, `/sync/*`, `/webhooks/*`) are handled by Bearer tokens, JWTs, or HMAC — not Cloudflare Access.

Pipeline 2: Xano Email/Password Auth (End-User Layer)

Scope: Direct email/password registration and login. Xano is the sole source of truth for user identity and password hashes.

Signup Flow

```
POST /auth/signup
  |
  +-- Validate email + password (8+ chars)
  +-- Require consent_tos + consent_privacy
  +-- xanoCreate(storefront_users)
  +-- hashPassword() --> PBKDF2 100k iterations, SHA-256
  +-- xanoCreate(user_claims) + consent_records
  +-- issueToken() --> Worker JWT (HS256, 7d TTL)
  +-- == Real-time sync ==
  |     +-- syncSingleUserToWebflow()
  |     +-- Shopify tagsAdd + metafields
  |     +-- pushToGA4() --> user properties
  |     +-- pushToAdobeAEP() --> XDM profile (SHA-256 hashed PII)
  +-- Return { token, user }
```

Login Flow

```
POST /auth/login
  |
  +-- xanoSearch(storefront_users, { email })
  +-- verifyPassword() --> PBKDF2 compare
  +-- issueToken() --> Worker JWT
  +-- Return { token, user }
```

Real-Time Logging (Signup)

DESTINATION	DATA WRITTEN	TRIGGER
Xano `storefront_users`	Full user record (email, name, provider, password_hash)	Immediate
Xano `user_claims`	Consent flags, provider metadata	Immediate
Xano `consent_records`	Audit entry (type, action, method, user_agent, timestamp)	Immediate
Webflow CMS	Customer item (name, email, tags, consent status)	Immediate
Shopify	Customer tags + metafields (crm_status, crm_tier, crm_tags)	Immediate
GA4	User properties (crm_status, crm_tier, consent_marketing)	Immediate
Adobe AEP	XDM profile with SHA-256 hashed email in identityMap	Immediate (if enabled)

Pipeline 3: Google OAuth (Federated Identity)

Scope: "Sign in with Google" via OpenID Connect authorization code flow.

Login Initiation

```
GET /auth/google/login
  |
  +-- Generate oauth_state:{uuid} --> KV (10min TTL, includes shop)
  +-- Redirect --> accounts.google.com/o/oauth2/v2/auth
        +-- scope: openid email profile
        +-- redirect_uri: /auth/google/callback
```

Callback Processing

```
GET /auth/google/callback?code=xxx&state=xxx
  |
  +-- Verify state matches KV (single-use, delete after)
  +-- Exchange code --> Google token endpoint
  +-- Decode ID token --> { sub, email, name, picture }
  +-- findOrCreateUser()
  |     +-- xanoSearch(storefront_users, { email })
  |     +-- If exists: update google_sub, avatar_url
  |     +-- If new: xanoCreate() with provider: "google"
  +-- xanoUpdate(user_claims, { google_sub, oidc_provider: "google" })
  +-- issueToken() --> Worker JWT (HS256, 7d TTL)
  +-- == Real-time logging ==
  |     +-- consent_records --> { method: "google_oauth", action: "granted" }
  |     +-- syncSingleUserToWebflow() --> CMS item create/update
  |     +-- Shopify --> customer create/update + tags + metafields
  |     +-- pushToGA4() --> user properties (provider: google)
  |     +-- pushToAdobeAEP() --> XDM profile with identityMap
  +-- Redirect --> AUTH_REDIRECT_ORIGIN with token cookie
```

Real-Time Logging (Google OAuth)

DESTINATION	DATA WRITTEN	TRIGGER
Xano `storefront_users`	google_sub, avatar_url, full_name, email	On callback
Xano `user_claims`	google_sub, oidc_provider: "google"	On callback
Xano `consent_records`	method: "google_oauth", action: "granted", user_agent, timestamp	On callback
Webflow CMS	Customer item with provider: "google", avatar, tags	On callback
Shopify	Customer tags (crm_status) + metafields	On callback
GA4	User properties: provider=google, crm_status, crm_tier, crm_tags	On callback
Adobe AEP	XDM profile: ECID + SHA-256(email), no raw PII	On callback (if enabled)

OAuth State Security

PARAMETER	STORAGE	TTL	PURPOSE
`oauth_state:{uuid}`	KV	10 min	CSRF protection, includes shop domain
Single-use	Deleted on callback	--	Prevents replay attacks
UUID-keyed	No global collisions	--	Multi-tenant safe

Pipeline 4: Shopify Customer Account OIDC (PKCE)

Scope: "Sign in with Shopify" via OAuth 2.0 with PKCE (no client secret required).

Login Initiation

```
GET /auth/shopify/login
  |
  +-- Generate PKCE code_verifier --> KV (pkce:{state}, 5min TTL)
  +-- Generate oauth_state:{uuid} --> KV (10min TTL, includes shop)
  +-- code_challenge = SHA-256(code_verifier) --> base64url
  +-- Redirect --> shopify.com/{shop_id}/auth/oauth/authorize
        +-- scope: openid email customer-account-api:full
        +-- code_challenge_method: S256
        +-- redirect_uri: /auth/shopify/callback
```

Callback Processing

```
GET /auth/shopify/callback?code=xxx&state=xxx
  |
  +-- Verify state matches KV (single-use, delete after)
  +-- Retrieve code_verifier from KV (pkce:{state})
  +-- Exchange code + code_verifier --> Shopify token endpoint
  |     +-- Returns access_token + id_token
  +-- Decode ID token --> { sub (shopify_customer_gid), email }
  +-- findOrCreateUser()
  |     +-- xanoSearch(storefront_users, { email })
  |     +-- If exists: update shopify_customer_gid
  |     +-- If new: xanoCreate() with provider: "shopify"
  +-- xanoUpdate(user_claims, {
  |     shopify_oidc_sub, oidc_provider: "shopify",
  |     shopify_customer_access_token
  |   })
  +-- issueToken() --> Worker JWT (HS256, 7d TTL)
  +-- == Real-time logging ==
  |     +-- consent_records --> { method: "shopify_oidc", action: "granted" }
  |     +-- syncSingleUserToWebflow() --> CMS item with shopify-customer-id
  |     +-- Shopify Admin API --> tagsAdd + metafields
  |     +-- pushToGA4() --> user properties (provider: shopify)
  |     +-- pushToAdobeAEP() --> XDM profile with identityMap
  +-- Redirect --> AUTH_REDIRECT_ORIGIN with token cookie
```

Real-Time Logging (Shopify OIDC)

DESTINATION	DATA WRITTEN	TRIGGER
Xano `storefront_users`	shopify_customer_gid, email, full_name	On callback
Xano `user_claims`	shopify_oidc_sub, oidc_provider, shopify_customer_access_token	On callback
Xano `consent_records`	method: "shopify_oidc", action: "granted", user_agent, timestamp	On callback
Webflow CMS	Customer item with shopify-customer-id, tags, consent	On callback
Shopify	Customer tags (crm_status, crm_tier) + metafields (crm_tags)	On callback
GA4	User properties: provider=shopify, crm_status, crm_tier, crm_tags, consent_*	On callback
Adobe AEP	XDM profile: ECID + SHA-256(email) in identityMap, no raw PII	On callback (if enabled)

PKCE + OAuth State Security

PARAMETER	STORAGE	TTL	PURPOSE
`pkce:{state}`	KV	5 min	PKCE code_verifier for S256 challenge
`oauth_state:{uuid}`	KV	10 min	CSRF protection, includes shop domain
Single-use	Both deleted on callback	--	Prevents replay attacks
UUID-keyed	No global collisions	--	Multi-tenant safe

Pipeline 5: Bearer Token /

ADMIN_KEY (API Admin Layer)

Scope: All admin, sync, and platform config endpoints.

```
POST /admin/*, /sync/*, /config, /platform/config
  |
  +-- resolveTenantId() --> tenant from ?shop= or header
  +-- verifyBearerToken(env, request, cfg.adminKey)
  |     +-- Check 1: tenant admin_key (per-tenant, if set)
  |     +-- Check 2: ADMIN_KEY env var (platform-level)
  |     +-- Check 3: SHOPIFY_APP_SECRET (app install flow)
  +-- Execute admin operation
```

Protected Endpoints

ROUTE	AUTH	PURPOSE
`POST /config`	Bearer	Write tenant config to KV
`GET /admin/tenants`	Bearer	List registered tenants (with region filter)
`POST /admin/provision-region`	Bearer	Create Xano tables for tenant
`GET/POST /platform/config`	Bearer	Read/write shared platform credentials
`POST /admin/init-tag-system`	Bearer	Initialize CRM tag tables
`POST /sync/customers`	Bearer	Trigger manual customer sync
`POST /sync/webflow`	Bearer	Trigger Webflow CMS sync

Pipeline 6: Shopify HMAC Webhooks

Scope: Inbound webhooks from Shopify (customer events + GDPR compliance).

```
POST /webhooks/customer-update, /webhooks/customer-create
  |
  +-- verifyShopifyHmac() --> validates X-Shopify-Hmac-SHA256
  +-- xanoSearch() or xanoCreate(storefront_users)
  +-- == Real-time sync ==
  |     +-- syncSingleUserToWebflow()
  |     +-- pushToGA4() --> user properties update
  |     +-- pushToAdobeAEP() --> XDM profile (if enabled)
  +-- Return 200


POST /gdpr/customer-redact, /gdpr/data-request, /gdpr/shop-redact
  |
  +-- verifyShopifyHmac() --> GDPR compliance verification
  +-- Execute GDPR operation (anonymize / compile / acknowledge)
  +-- Return 200
```

JWT Session Management

All four user-facing pipelines (email/password, Google, Shopify, webhook-created) converge on the same JWT issuance:

PROPERTY	VALUE
Issuer	Worker (not Xano, not Cloudflare)
Algorithm	HS256 (HMAC-SHA256)
Secret	`cfg.jwtSecret` (wrangler secret)
TTL	7 days (604,800 seconds)
Payload	`{ sub, email, name, provider, iat, exp }`
Delivery	`httpOnly`, `Secure`, `SameSite=None` cookie
Verification	`authenticateRequest()` on all `/ucp/*`, `/auth/me`, `/auth/profile`, `/tags/*`, `/segment/*`

Real-Time Logging Matrix

Every auth event triggers immediate writes to up to 7 systems:

EVENT	XANO USERS	XANO CLAIMS	XANO CONSENT	WEBFLOW CMS	SHOPIFY	GA4	ADOBE AEP
Email signup	W	W	W	W	W	W	W
Email login	R	--	--	--	--	--	--
Google OAuth	W	W	W	W	W	W	W
Shopify OIDC	W	W	W	W	W	W	W
Shopify webhook	W	--	--	W	--	W	W
Tag change (UCP)	--	--	--	W	W	W	W
Consent update	--	W	W	--	--	W	--
Password reset	R	--	--	--	--	--	--
Account delete	D	D	W	D	--	--	--

W = Write, R = Read, D = Delete, -- = No interaction

Security Boundaries

LAYER	MECHANISM	VALIDATES	SCOPE
Cloudflare Access	Edge OTP email	Browser identity	Admin UI only
ADMIN_KEY Bearer	Shared secret comparison	API caller identity	Admin + sync endpoints
Per-tenant admin_key	Tenant-scoped secret	Tenant-level API access	Checked before platform key
Worker JWT	HS256 signature + expiry	End-user session	User-facing endpoints
Shopify HMAC	SHA-256 request signature	Webhook integrity	Inbound webhooks + GDPR
OAuth state	UUID-keyed KV nonce	CSRF prevention	Google + Shopify OAuth
PKCE	S256 code challenge	Authorization code binding	Shopify OIDC only

No cross-system token validation exists. Each layer owns its auth independently.

Glossary: What These Terms Mean

Technical security terms explained in plain language.

PKCE (Proof Key for Code Exchange)

What it is: A security add-on for login flows that prevents someone from stealing your login code mid-transit.

How it works in plain terms: When you click "Sign in with Shopify," the system creates a secret answer and a matching puzzle. It sends only the puzzle to Shopify. When Shopify sends back your login code, the system proves it's the

original requester by revealing the secret answer. If an attacker intercepted the login code, they can't use it because they don't have the secret answer.

Why it matters: Older login flows relied on a shared password (client secret) between the app and Shopify. PKCE eliminates that — the secret is created fresh for every single login attempt and is never transmitted over the network. This is especially important for apps that run in browsers or on edge servers where storing long-lived secrets is risky.

In CRM Sync: Used for "Sign in with Shopify" (Pipeline 4). The code_verifier is stored in KV for 5 minutes, and the SHA-256 code_challenge is sent to Shopify. On callback, the verifier is retrieved, verified, and deleted — single use only.

TLS (Transport Layer Security)

What it is: The encryption that protects data as it moves between your browser and the server. It's the "S" in HTTPS.

How it works in plain terms: Every time your browser connects to CRM Sync (or any HTTPS site), TLS creates an encrypted tunnel. Everything that travels through that tunnel — passwords, tokens, customer data — is scrambled so that anyone intercepting the traffic sees only random noise. The lock icon in your browser address bar means TLS is active.

Why it matters: Without TLS, anyone on the same network (coffee shop Wi-Fi, corporate network, ISP) could read passwords, session tokens, and customer data in plain text. TLS is the baseline — every other security mechanism in this document assumes TLS is in place.

In CRM Sync: All worker endpoints run on Cloudflare's edge network, which enforces TLS 1.2+ on every connection. All API calls between the worker and external services (Xano, Shopify, Google, GA4, Adobe, Resend) also use HTTPS/TLS. No data ever travels unencrypted.

OIDC (OpenID Connect)

What it is: A standard protocol that lets users sign in to your app using an account they already have — like Google or Shopify — without sharing their password with you.

How it works in plain terms: When a user clicks "Sign in with Google," they're redirected to Google's login page. Google verifies who they are, then sends back a signed ID card (called an "ID token") that says "this person is jane@example.com and we verified it." Your app trusts this ID card because it's signed by Google. The user's Google password never touches your system.

Why it matters: Users don't need to create yet another password (which they'll probably reuse from another site). You don't need to store or protect passwords for these users. And if Google or Shopify detects suspicious activity on the user's account, they can revoke access — something you can't do with a simple email/password.

In CRM Sync: Both Google (Pipeline 3) and Shopify (Pipeline 4) use OIDC. The worker receives an ID token, extracts the user's identity (email, name, profile picture), and creates or links their CRM account. The worker never sees or stores the user's Google or Shopify password.

JWT (JSON Web Token)

What it is: A compact, digitally signed pass that proves who a user is. Think of it as a tamper-proof wristband at an event — you get it once at the door (login), and it gets you into everything without showing ID again.

How it works in plain terms: After you log in (by any method — email, Google, or Shopify), the worker creates a small packet containing your user ID, email, and name, then signs it with a secret key. This signed packet (the JWT) is stored in your browser as a cookie. Every time you access a protected page, the browser sends the JWT back, and the worker verifies the signature hasn't been tampered with and the token hasn't expired.

In CRM Sync: The worker (not Xano, not Cloudflare) issues all JWTs using HS256 signing. Tokens expire after 7 days. They're delivered as `httpOnly` cookies so JavaScript on the page can't read or steal them.

HMAC (Hash-Based Message Authentication Code)

What it is: A way to verify that a message (like a webhook from Shopify) actually came from who it claims to come from and wasn't altered in transit.

How it works in plain terms: Shopify and the worker share a secret key. When Shopify sends a webhook, it creates a fingerprint of the message body using that secret key and attaches it as a header. The worker creates its own fingerprint using the same key and message body. If the two fingerprints match, the message is authentic and unaltered.

In CRM Sync: All Shopify webhooks (customer create/update) and GDPR compliance requests are verified via HMAC-SHA256. If the fingerprint doesn't match, the request is rejected with a 401.

Zero Trust

What it is: A security philosophy that says "never trust, always verify" — even if you're already inside the network. Every request must prove it's authorized, every time.

How it works in plain terms: Traditional security is like a building with a locked front door — once you're inside, you can go anywhere. Zero Trust is like a building where every room has its own lock and every person must badge in at every door, even if they just badged in at the room next door.

In CRM Sync: Cloudflare Access implements Zero Trust for admin pages. Even though you might have the worker URL, you still need to verify your email via one-time password before accessing `/setup`, `/settings`, or `/onboarding`. API endpoints use their own independent auth (Bearer tokens, JWTs, HMAC) — no single credential grants access to everything.

AI-Native Security Requirements

Modern applications that integrate AI services (LLM APIs, agent frameworks, MCP servers) introduce security concerns that traditional auth models don't address. CRM Sync touches AI through the Anthropic SDK (shop-chat-agent) and MCP server integrations. These requirements apply to any AI-augmented features.

1. Credential Isolation for AI Agents

Requirement: AI agents and MCP servers must never have direct access to production credentials (API keys, admin tokens, OAuth secrets).

How CRM Sync addresses this:

- MCP servers receive scoped environment variables (`SHOPIFY_ACCESS_TOKEN`, `MYSHOPIFY_DOMAIN`) — not the full credential set
- The worker's `ADMIN_KEY`, `JWT_SECRET`, and `SHOPIFY_APP_SECRET` are wrangler secrets, never exposed to client-side code or agent contexts
- `getPublicCrmConfig()` strips all API keys and secrets before injecting config into any embed or client-facing response

What to watch for: If an AI agent is given tool access to `POST /config`, it could overwrite credentials. Admin endpoints must require explicit human-issued Bearer tokens, not agent-delegated ones.

2. Prompt Injection & Data Boundary Enforcement

Requirement: User-generated content (names, tags, form inputs) must never be interpreted as instructions by AI systems.

How CRM Sync addresses this:

- All user inputs are stored as data in Xano (structured fields, not free-text prompts)
- CRM tag names are slug-sanitized (`slugify()`) before storage — no executable content
- The worker treats all inbound data as untrusted: HTML-escaped in embeds, parameterized in API calls
- No user-supplied text is passed to LLM prompts without explicit sanitization boundaries

What to watch for: If a future feature sends CRM tag names or customer notes to an LLM for summarization, those fields become prompt injection vectors. Always wrap user-sourced content in explicit delimiters and instruct the model to treat it as data.

3. Token Scope & Lifetime for AI Workflows

Requirement: AI agents operating on behalf of users must use short-lived, narrowly scoped tokens — not long-lived admin keys.

How CRM Sync addresses this:

- Worker JWTs (7-day TTL) carry only `{ sub, email, name, provider }` — no admin privileges
- OAuth state tokens are single-use and expire in 5-10 minutes
- PKCE code verifiers are per-session and never reused
- Per-tenant admin keys scope access to a single tenant, not the whole platform

Best practice for AI features: If an AI agent needs to call worker endpoints on behalf of a user, issue a short-lived scoped token (1-hour max) with explicit permissions (e.g., read-only CRM tags). Never pass the platform `ADMIN_KEY` to an agent context.

4. Audit Trail for AI-Initiated Actions

Requirement: Every action taken by an AI agent must be logged with the same rigor as human-initiated actions, including the agent identity and the human who authorized it.

How CRM Sync addresses this:

- `consent_records` logs every consent change with method, timestamp, user_agent, and session ID
- Tag changes via `/ucp/tags` log the `source` field (`shopify`, `ucp`, `admin`, `system`)
- All admin operations require Bearer auth, so the actor (human or agent) is identifiable by the key used

What to add for AI features: Introduce an `agent` source value for the `source` field in `user_tag_map` and `consent_records`. Log the agent type (e.g., `mcp:shopify`, `chat-agent`) alongside the authorizing user's identity. This creates an unbroken chain: human → agent → action → audit entry.

5. MCP Server Security Boundaries

Requirement: MCP (Model Context Protocol) servers bridging AI agents to external APIs must enforce the same auth boundaries as direct API access.

How CRM Sync addresses this:

- MCP servers are configured with `--scope project` or `--scope user`, limiting their reach
- Each MCP server receives only the environment variables it needs (principle of least privilege)
- The Shopify MCP server gets `SHOPIFY_ACCESS_TOKEN` and `MYSHOPIFY_DOMAIN` — not the worker's `ADMIN_KEY` or `JWT_SECRET`

What to watch for:

- An MCP server with `write_customers` scope can modify customer data in Shopify — this should be gated behind explicit user confirmation in the agent workflow
- If multiple MCP servers are active simultaneously, ensure they can't read each other's credentials via shared environment
- MCP tool calls should be logged (Claude Code does this automatically via hooks) so that unexpected API usage is traceable

6. Data Minimization in AI Contexts

Requirement: AI agents should receive only the data they need to complete the task — not full customer records, credential sets, or PII.

How CRM Sync addresses this:

- `GET /config` masks all credentials (first 4 + last 4 chars) — safe for diagnostic prompts
- Adobe AEP integration hashes all PII with SHA-256 before transmission — the raw email never leaves the worker
- Embed HTML (`/embed/footer`, `/embed/compliance`) receives only public config via `getPublicCrmConfig()`

Best practice for AI features: If an AI agent needs customer context for personalization, pass only the customer's tag slugs and consent status — not their email, name, or Shopify ID. Use the `crm_tags` and `consent_*` fields, which are non-identifying.
